



PROJECT-BASED LEARNING

PHASE-I EVALUATION

VitalRoute: A Smart Multi-Specialty Hospital Management & Emergency Routing System

Team ID: DSCPP-III-2026-T120

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T120
Semester: 3rd (*confirm*)
Domain: *Healthcare / Hospital Management Systems*

Team Members

1. Nakshatra Agarwal – 2510370215 (*ML1*)
2. Devesh Saini – 2510370251 (*ML4*)
3. Vasav Sharma – 25110224030 (*E*)

Mentor

Ms. Pratibha Waldia

Interactions completed: 3

Problem Statement & Motivation

Problem Statement

In a multispecialty hospital, patients cannot be treated in simple arrival order – a critical case and a routine case need fundamentally different handling – yet emergency triage, bed/OT/ward allocation, and billing are still frequently manual, staff-judgement-driven processes in small and mid-sized hospitals.

Motivation

- ▶ Manual ward-checking and paper records slow down routine care
- ▶ Minutes lost finding an ICU bed or free OT can affect outcomes
- ▶ Basic hospital software rarely triages by severity automatically

Target Users / Stakeholders

- ▶ Patients – emergency and outpatient (OPD)
- ▶ Doctors, nurses, and hospital admin staff
- ▶ Benefit: faster, severity-correct, auditable care

Objectives

Project Objectives

1. Design and implement a working hospital-management simulator covering registration, triage, routing, records, and billing.
2. Implement severity-based emergency triage and graph-based nearest-resource (ICU/OT/ward) routing.
3. Integrate Data Structures (C) and OOP (C++) concepts meaningfully in one coherent system.
4. Verify correctness of triage ordering, routing, and billing calculations through test cases.
5. Demonstrate the console-based system now; GUI and role-based reporting are later-phase extensions.

Key Objective: Automatically route every simulated patient to the nearest appropriate resource and produce an accurate, insurance-adjusted bill.

Proposed Solution

Proposed Approach

1. Patient / Staff Registration (Input)
2. Triage & Routing (Priority Queue + Graph)
3. Consultation & Diagnosis Logic
4. Records & Billing Storage (Hashing, AVL, Files)
5. Bill & Discharge Output

Key Features

- ▶ Severity-based triage via a shared Common Central ER to ICU / OT / Ward
- ▶ Circular-queue OT scheduling; dequeue-based ambulance/transfer requests
- ▶ Multispecialty referral loop across departments
- ▶ Doctor's chronological visit log (diagnosis, verdict, medicines)
- ▶ Automatic + manual billing with insurance deduction
- ▶ AVL date-range search; binary search doctor lookup
- ▶ Patient & Doctor portals (planned GUI)

This is a simulator for academic demonstration – it does not connect to any real hospital, patient, or payment data.

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	Priority Queue, Circular Queue, Dequeue, Graph (Shortest Path), Hashing, AVL / B+ Tree, Doubly Linked List, Stack, Binary Search, Sorting	Emergency Triage, ER Routing, Records & Search
OOPs with C++	Class Hierarchy & Inheritance, Polymorphism, Operator Overloading, Exception Handling, File Handling, STL	Patient/Staff Hierarchy, Billing Engine

Requirement: Demonstrate meaningful integration of both subjects in **one** project.

System Architecture / Workflow



Major Components

- ▶ Patient & Staff Management
- ▶ Emergency Triage & ER Routing
- ▶ Consultation & Billing Engine

Data / Control Flow

- ▶ Severity decides queue order, not arrival time
- ▶ Graph search finds the nearest free ICU / OT / ward
- ▶ Charges accumulate automatically; insurance is deducted before the final bill

Technology Stack & Methodology

Technology Stack

- ▶ Programming: C (data-structure engine) & C++ (OOP layer)
- ▶ Storage: No external DBMS – file-based (`fstream`) with hashed & AVL/B+-tree indexing
- ▶ Framework: Standard C++ Library (STL); Qt (*planned, Phase III GUI*)
- ▶ IDE: *VS Code / Code::Blocks (GCC/G++)*
- ▶ Version Control: *Git & GitHub*

Development Methodology

1. Requirement Analysis
2. System Design
3. Implementation
4. Testing
5. Evaluation

No real DBMS is used by design – triage, routing, and billing are demonstrated through data structures and file handling, not a database layer.

Initial Progress & Team Contribution

Progress Achieved

- ▶ Problem study & background research completed
- ▶ Requirements & feature list finalized
- ▶ System architecture & workflow designed
- ▶ DSA / OOP module mapping completed
- ▶ Implementation begins in Phase II

Individual Contribution

Member	Role	Contribution
Nakshatra Agarwal	Team Lead	<i>Emergency triage & ER routing logic; integration</i>
Devesh Saini	Developer	<i>Billing engine & OOP class design</i>
Vasav Sharma	Developer	<i>Records, file handling, indexing & testing</i>

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal & Design (current)
2. Phase-II: Core console implementation
3. Phase-III: Integration testing + Qt GUI

Expected Outcomes

- ▶ Working console prototype: triage, routing, billing
- ▶ Accurate, testable results against sample cases
- ▶ Practical model of real hospital operations

References

1. Reema Thareja, *Data Structures Using C*, Oxford University Press
2. E. Balagurusamy, *Object-Oriented Programming with C++*, McGraw Hill
3. cppreference.com – C++ & STL reference
4. [GeeksforGeeks](http://GeeksforGeeks.com) – DS/algorithm explanations
5. Qt Documentation, doc.qt.io

Thank You
Questions & Suggestions